Practice Problem — Non-Linear Optimization
Gradient Inversion Attack in Federated Learning

Non-Linear Optimization Course

May 30, 2025

Background

Federated Learning (FL) keeps training data on client devices and shares only gradients $\mathbf{g} = \nabla_{\boldsymbol{\theta}} \mathcal{L}\Big(f_{\boldsymbol{\theta}}(\mathbf{x}), y\Big)$. Yet, *gradient inversion* can reconstruct $(\mathbf{x}, y)$ from $\mathbf{g}$ and the current model $f_{\boldsymbol{\theta}}$ [1, 2, 3, 4].

Optimization Problem

Given:

- model parameters $\boldsymbol{\theta}$ (2-layer MLP, trained on MNIST);
- a leaked gradient vector $\mathbf{g}^\star$ from a *single* unknown sample.

Recover an image $\hat{\mathbf{x}} \in [0,1]^{28\times 28}$ and label $\hat{y} \in \{0, \dots, 9\}$ by minimizing

$$\min_{\mathbf{x}\in[0,1]^{784},\, y\in\{0,\dots,9\}} \left\| \nabla_{\boldsymbol{\theta}} \mathcal{L}\Big(f_{\boldsymbol{\theta}}(\mathbf{x}), y\Big) - \mathbf{g}^\star \right\|_2^2 + \lambda \|\mathbf{x}\|_2^2.$$

Tasks

1. Implement L-BFGS, Adam, or both, to optimize $\mathbf{x}$ (continuous) and y (use one-hot logits or the iDLG pseudo-label trick).
2. Run at least five random initializations and keep the best solution.
3. Plot objective value over iterations and pixel MSE versus ground truth (provided for grading).
4. Study sensitivity to λ and optimizer hyper-parameters.

Deliverables (100 pts)

- **Code (40 pts)** — reproducible, with `requirements.txt`.
- **Report (40 pts)** — max 4 pages: formulation, method, results, discussion.
- **Slides (20 pts)** — 5 min lightning talk.

Starter Code (PyTorch)

```
import torch, torch.nn as nn, torch.optim as optim

# resources provided
theta = torch.load("mlp_mnist_weights.pt")
g_star = torch.load("one_sample_gradient.pt")

model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 128), nn.ReLU(),
    nn.Linear(128, 10)
)
with torch.no_grad():
    for p, src in zip(model.parameters(), theta):
        p.copy_(src)

loss_fn = nn.CrossEntropyLoss()

def reconstruct(lambda_reg=1e-4, iters=300, lr=0.1, seed=0):
    torch.manual_seed(seed)
    x_hat = torch.rand(1, 1, 28, 28, requires_grad=True)
    y_logits = torch.zeros(1, 10, requires_grad=True)
    opt = optim.LBFGS([x_hat, y_logits], lr=lr, max_iter=20)

    for _ in range(iters):
        def closure():
            opt.zero_grad()
            y_prob = y_logits.softmax(dim=-1)
            loss = loss_fn(model(x_hat), y_prob)
            grads = torch.autograd.grad(loss, model.parameters(),
                                        create_graph=True)
            grad_diff = torch.cat([g.view(-1) for g in grads]) -
               g_star
            obj = grad_diff.pow(2).sum() + lambda_reg * x_hat.pow
               (2).sum()
            obj.backward()
            return obj
        opt.step(closure)
    return x_hat.detach(), y_logits.argmax(dim=-1).item()
```

Hints

- Normalise $\mathbf{g}^\star$ and use gradient clipping.
- Use multiple restarts—the landscape has many poor local minima.
- TV-loss or generative priors (GAN latents) can refine images but enlarge the search space.

Ethics Notice

Apply this attack **only** to the classroom model and data.

References

[1] L. Zhu, Z. Liu, S. Han. *Deep Leakage from Gradients*. NeurIPS 2019.

[2] L. Zhu, S. Han. *iDLG: Improved Deep Leakage from Gradients*. arXiv:2001.02610 (2020).

[3] A. Ribeiro *et al. DLG-FB: Feedback-Blending Gradient Inversion*. arXiv:2409.17767 (2024).

[4] P. Guo *et al. Survey of Gradient Inversion Attacks*. 2025.